

Labolatorium 13

June 17, 2026

Jakub Orchowski, s223281

1 CEL ĆWICZENIA

Filtrowanie sygnałów jedno- i dwuwymiarowych.

```
[1]: import os
from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
from scipy.io import wavfile
from scipy.signal import firwin, freqz, lfilter, convolve2d
from scipy.ndimage import median_filter
%matplotlib inline
```

2 Zadania

```
[2]: def asset_path(name: str) -> str:
    """Return path to asset; works from repo root or lab13 directory."""
    repo_path = Path('lab13') / name
    if repo_path.exists():
        return str(repo_path)
    return name

def output_path(name: str) -> str:
    """Return output path inside lab13/ when run from repo root, otherwise_
    ↪ current dir."""
    repo_path = Path('lab13') / name
    if Path('lab13').is_dir():
        return str(repo_path)
    return name

def normalize_image(x: np.ndarray) -> np.ndarray:
    """Normalize array to [0, 1] for display."""
```

```

    if x.max() == x.min():
        return np.zeros_like(x, dtype=float)
    return (x - x.min()) / (x.max() - x.min())

def plot_signal_fft(signal: np.ndarray, fs: int, title: str = "Amplituda FFT") → None:
    """Plot amplitude spectrum of a 1D signal using fftshift."""
    n = len(signal)
    freqs = np.fft.fftshift(np.fft.fftfreq(n, 1 / fs))
    spectrum = np.abs(np.fft.fftshift(np.fft.fft(signal)))
    fig, ax = plt.subplots(figsize=(10, 4))
    ax.plot(freqs, spectrum)
    ax.set_title(title)
    ax.set_xlabel("Częstotliwość [Hz]")
    ax.set_ylabel("Amplituda")
    ax.set_xlim(-fs / 2, fs / 2)
    plt.tight_layout()
    plt.show()

def plot_filter_response(b: np.ndarray, worN: int = 512, title: str = "Odpowiedź częstotliwościowa filtru") → None:
    """Plot amplitude frequency response of a 1D FIR filter using freqz."""
    w, h = freqz(b, a=1, worN=worN, whole=True)
    freqs = np.fft.fftshift(w)
    response = np.fft.fftshift(np.abs(h))
    fig, ax = plt.subplots(figsize=(10, 4))
    ax.plot(freqs / np.pi, response)
    ax.set_title(title)
    ax.set_xlabel("Częstotliwość znormalizowana")
    ax.set_ylabel("Amplituda")
    ax.grid(True)
    plt.tight_layout()
    plt.show()

def freqz2(h: np.ndarray, shape: tuple = (256, 256)) → np.ndarray:
    """Compute 2D frequency response of a kernel (MATLAB freqz2 equivalent)."""
    h_big = np.fft.fft2(h, s=shape)
    return np.fft.fftshift(h_big)

def plot_2d_freq_response(h: np.ndarray, title: str = "Odpowiedź częstotliwościowa 2D") → None:
    """Display 2D filter frequency response as image and surface."""
    h_amp = np.abs(freqz2(h))

```

```

size = h_amp.shape[0]
extent = (-np.pi, np.pi, -np.pi, np.pi)
x = np.linspace(-np.pi, np.pi, size)
y = np.linspace(-np.pi, np.pi, size)
x_grid, y_grid = np.meshgrid(x, y)

fig = plt.figure(figsize=(12, 5))
ax1 = fig.add_subplot(1, 2, 1)
im = ax1.imshow(h_amp, cmap="viridis", extent=extent, origin="lower")
ax1.set_title(title)
ax1.set_xlabel(r"$\omega_x$")
ax1.set_ylabel(r"$\omega_y$")
fig.colorbar(im, ax=ax1)

ax2 = fig.add_subplot(1, 2, 2, projection="3d")
ax2.plot_surface(x_grid, y_grid, h_amp, cmap="viridis", edgecolor="none")
ax2.set_title(f"{title} - powierzchnia")
ax2.set_xlabel(r"$\omega_x$")
ax2.set_ylabel(r"$\omega_y$")
ax2.set_zlabel("Amplituda")
plt.tight_layout()
plt.show()

def filter2d(image: np.ndarray, kernel: np.ndarray) -> np.ndarray:
    """Apply a 2D kernel to an image (MATLAB filter2 equivalent)."""
    return convolve2d(image.astype(np.float64), kernel, mode="same",
        ↪boundary="symm")

def show_row(images: list, titles: list, cmap: str = "gray") -> None:
    """Display a row of images."""
    n = len(images)
    fig, axes = plt.subplots(1, n, figsize=(4 * n, 4))
    if n == 1:
        axes = [axes]
    for ax, img, title in zip(axes, images, titles):
        ax.imshow(img, cmap=cmap)
        ax.set_title(title)
        ax.axis("off")
    plt.tight_layout()
    plt.show()

```

2.1 Zadanie 1.

Dany jest zdyskretyzowany sygnał akustyczny icing2.wav (częstotliwość próbkowania 44100 Hz). Oblicz transformatę Fouriera (fft) tego sygnału i wyświetl jej amplitudę. Następnie przefiltruj syg-

nał filtrem uśredniającym o szerokości 100 próbek (a) oraz filtrem uzyskanym z polecenia `fir1(99, 0.1)` (b). Dla każdego przypadku zapisz wynik do pliku wav, wyświetl amplitudę odpowiedzi częstotliwościowej filtru oraz amplitudę FFT przefiltrowanego sygnału.

2.1.1 Oryginalny sygnał i jego widmo

```
[3]: fs, icing2 = wavfile.read(asset_path('icing2.wav'))
icing2 = icing2.astype(np.float64)
icing2_norm = icing2 / np.max(np.abs(icing2))

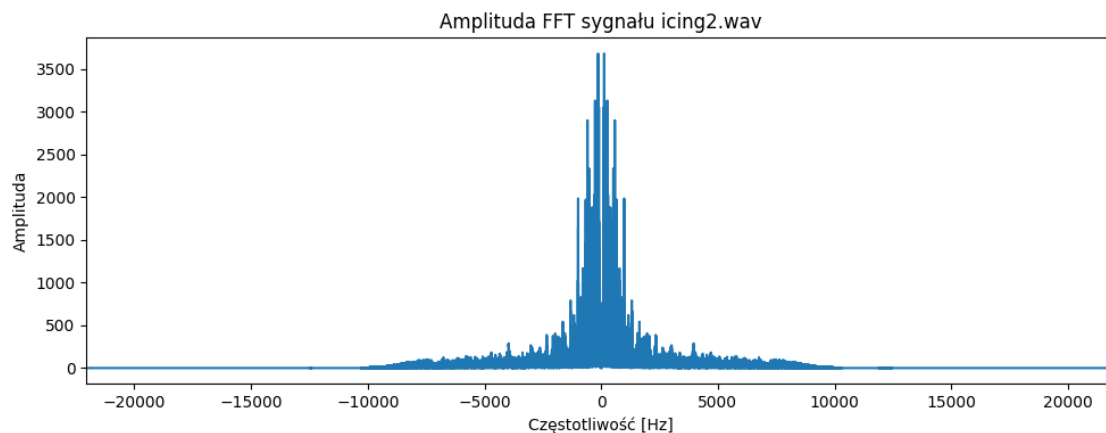
print(f"Częstotliwość próbkowania: {fs} Hz")
print(f"Liczba próbek: {len(icing2_norm)}")
print(f"Czas trwania: {len(icing2_norm) / fs:.3f} s")

plot_signal_fft(icing2_norm, fs, title="Amplituda FFT sygnału icing2.wav")
```

Częstotliwość próbkowania: 44100 Hz

Liczba próbek: 500000

Czas trwania: 11.338 s

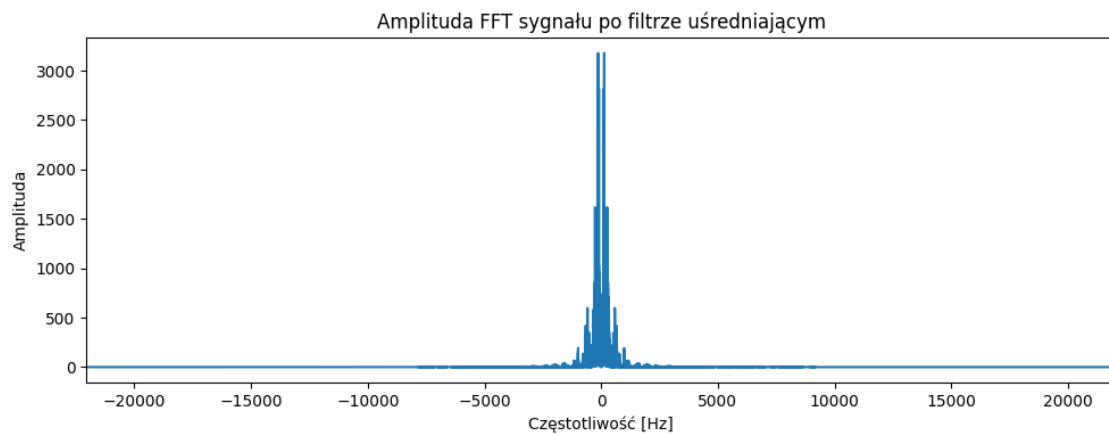
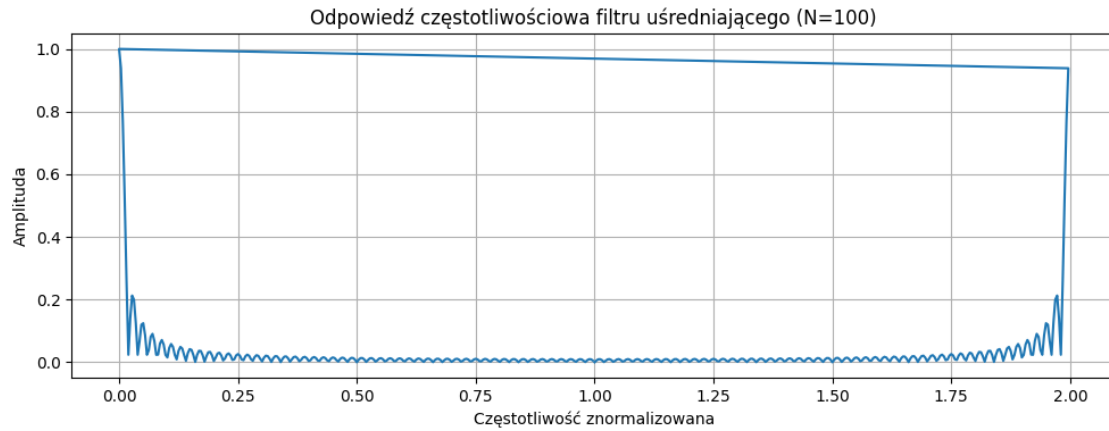


2.1.2 a) Filtr uśredniający 100 próbek

```
[4]: avg_kernel = np.ones(100) / 100
icing2_avg = lfilter(avg_kernel, 1, icing2_norm)

wavfile.write(output_path('icing2_avg.wav'), fs, (icing2_avg * 32767).astype(np.
    ↪int16))

plot_filter_response(avg_kernel, worN=512, title="Odpowiedź częstotliwościowa_
    ↪filtru uśredniającego (N=100)")
plot_signal_fft(icing2_avg, fs, title="Amplituda FFT sygnału po filtrze_
    ↪uśredniającym")
```

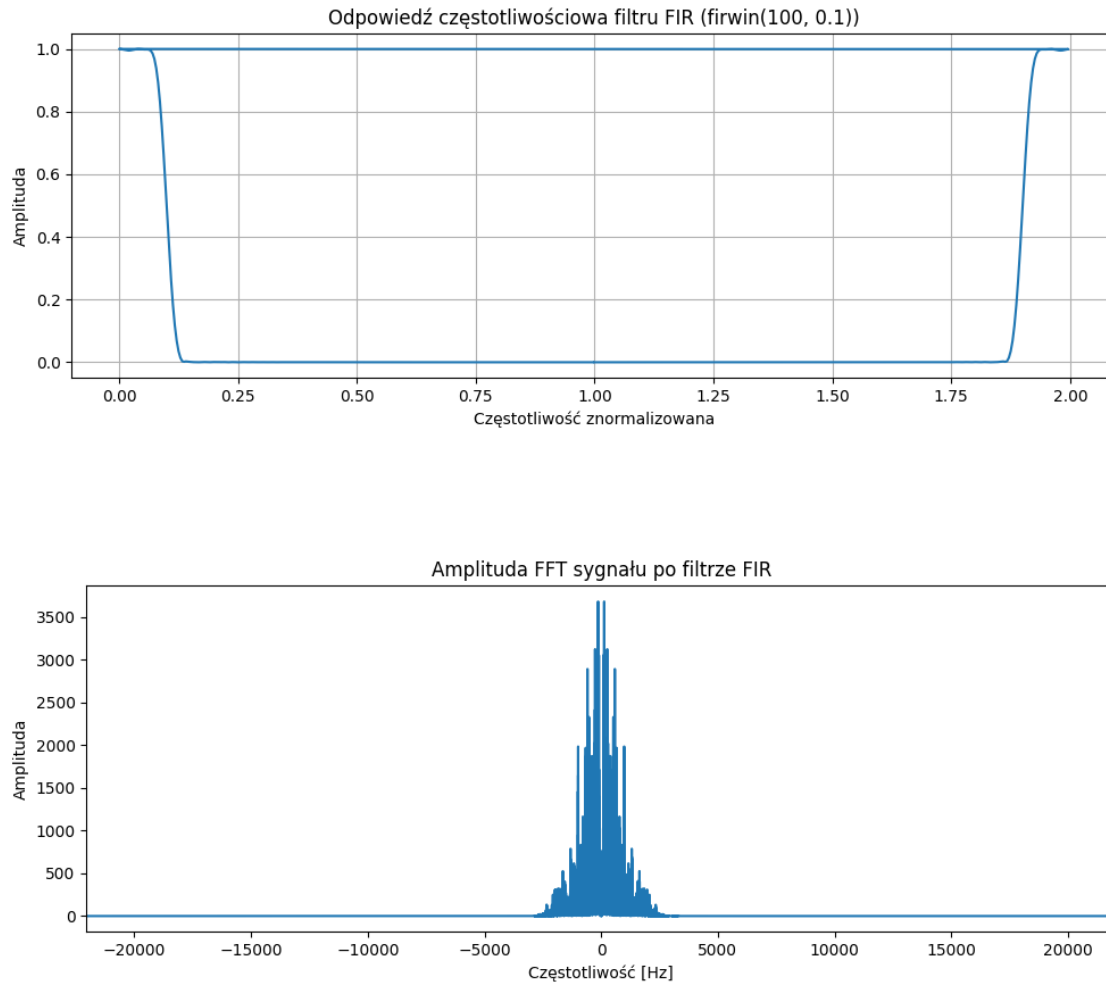


2.1.3 b) Filtr FIR uzyskany poleceniem `fir1(99, 0.1)`

```
[5]: fir_kernel = firwin(100, 0.1)
icing2_fir = lfilter(fir_kernel, 1, icing2_norm)

wavfile.write(output_path('icing2_fir.wav'), fs, (icing2_fir * 32767).astype(np.
    ↪int16))

plot_filter_response(fir_kernel, worN=512, title="Odpowiedź częstotliwościowa_
    ↪filtru FIR (firwin(100, 0.1))")
plot_signal_fft(icing2_fir, fs, title="Amplituda FFT sygnału po filtrze FIR")
```



2.1.4 Wnioski

Filtr uśredniający działa jak dolnoprzepustowy filtr sinc o wolno opadających listkach bocznych, co widoczne jest jako charakterystyczne wygaszanie wysokich częstotliwości. Filtr FIR projektowany funkcją `firwin` ma znacznie ostrzejsze zbocze i mniejsze listki boczne, dzięki czemu lepiej odcina składowe powyżej częstotliwości odcięcia. W obu przypadkach widmo sygnału wyjściowego zawiera mniej energii w wysokich częstotliwościach.

2.2 Zadanie 2.

Przetwórz obraz `LAKE.bmp` za pomocą dolnoprzepustowych filtrów uśredniających o symetrycznych jądrach 3×3 , 5×5 , 7×7 , a następnie o niesymetrycznych jądrach 3×5 , 3×11 , 3×19 . Wyświetl kształt odpowiedzi częstotliwościowych oraz przefiltrowane obrazy.

```
[6]: lake = np.array(Image.open(asset_path('LAKE.bmp')).convert('L'))

symmetric_sizes = [(3, 3), (5, 5), (7, 7)]
```

```

asymmetric_sizes = [(3, 5), (3, 11), (3, 19)]

def averaging_kernel(size: tuple) -> np.ndarray:
    """Create normalized averaging kernel of given shape."""
    return np.ones(size) / (size[0] * size[1])

print(f"Rozmiar obrazu LAKE: {lake.shape}")

```

Rozmiar obrazu LAKE: (256, 256)

```

[7]: # Symmetric averaging filters
fig_resp, axes_resp = plt.subplots(1, 3, figsize=(15, 4))
fig_img, axes_img = plt.subplots(1, 3, figsize=(15, 4))

for idx, size in enumerate(symmetric_sizes):
    kernel = averaging_kernel(size)
    filtered = filter2d(lake, kernel)

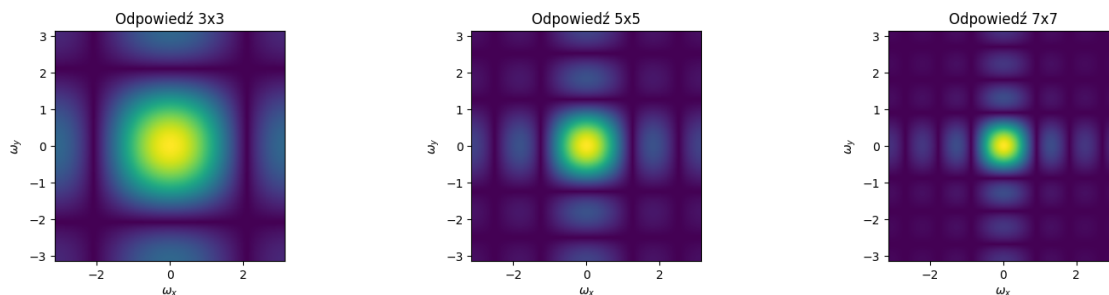
    axes_resp[idx].imshow(np.abs(freqz2(kernel)), cmap="viridis", extent=(-np.
↳ pi, np.pi, -np.pi, np.pi), origin="lower")
    axes_resp[idx].set_title(f"Odpowiedź {size[0]}x{size[1]}")
    axes_resp[idx].set_xlabel(r"$\omega_x$")
    axes_resp[idx].set_ylabel(r"$\omega_y$")

    axes_img[idx].imshow(filtered, cmap="gray")
    axes_img[idx].set_title(f"Filtr {size[0]}x{size[1]}")
    axes_img[idx].axis("off")

fig_resp.suptitle("Symetryczne filtry uśredniające - odpowiedzi_
↳ częstotliwościowe", y=1.02)
fig_img.suptitle("Symetryczne filtry uśredniające - obrazy po filtracji", y=1.
↳ 02)
fig_resp.tight_layout()
fig_img.tight_layout()
plt.show()

```

Symetryczne filtry uśredniające — odpowiedzi częstotliwościowe



Symetryczne filtry uśredniające — obrazy po filtracji



```
[8]: # Asymmetric averaging filters
fig_resp, axes_resp = plt.subplots(1, 3, figsize=(15, 4))
fig_img, axes_img = plt.subplots(1, 3, figsize=(15, 4))

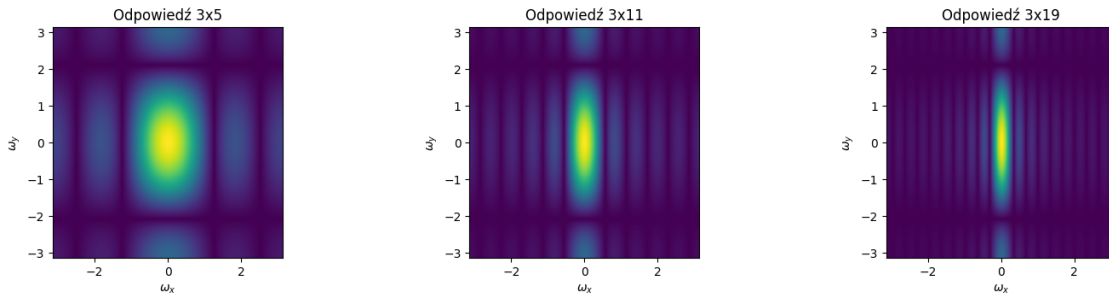
for idx, size in enumerate(asymmetric_sizes):
    kernel = averaging_kernel(size)
    filtered = filter2d(lake, kernel)

    axes_resp[idx].imshow(np.abs(freqz2(kernel)), cmap="viridis", extent=(-np.
    ↪pi, np.pi, -np.pi, np.pi), origin="lower")
    axes_resp[idx].set_title(f"Odpowiedź {size[0]}x{size[1]}")
    axes_resp[idx].set_xlabel(r"$\omega_x$")
    axes_resp[idx].set_ylabel(r"$\omega_y$")

    axes_img[idx].imshow(filtered, cmap="gray")
    axes_img[idx].set_title(f"Filtr {size[0]}x{size[1]}")
    axes_img[idx].axis("off")

fig_resp.suptitle("Niesymetryczne filtry uśredniające - odpowiedzi_
    ↪częstotliwościowe", y=1.02)
fig_img.suptitle("Niesymetryczne filtry uśredniające - obrazy po filtracji",
    ↪y=1.02)
fig_resp.tight_layout()
fig_img.tight_layout()
plt.show()
```


Niesymetryczne filtry uśredniające — odpowiedzi częstotliwościowe



Niesymetryczne filtry uśredniające — obrazy po filtracji



2.2.1 Wnioski

Zwiększenie rozmiaru jądra uśredniającego prowadzi do silniejszego rozmycia obrazu i węższej odpowiedzi częstotliwościowej w dziedzinie Fourier, co oznacza lepsze tłumienie wysokich częstotliwości. Filtry niesymetryczne rozmycają obraz silniej w kierunku większego wymiaru, co odzwierciedla się w wydłużonym kształcie odpowiedzi częstotliwościowej.

2.3 Zadanie 3.

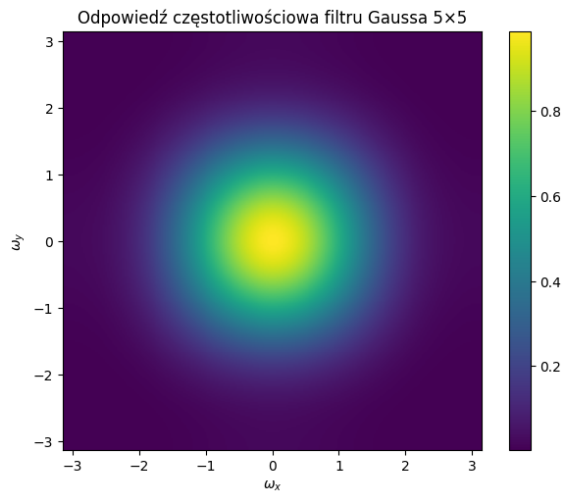
Przetwórz obraz LAKE.bmp dolnoprzepustowym filtrem Gaussa o zadanym jądrze 5×5 . Wyświetl kształt odpowiedzi częstotliwościowej oraz przefiltrowany obraz.

```
[9]: gaussian_kernel = np.array([
    [0.003, 0.013, 0.022, 0.013, 0.003],
    [0.013, 0.059, 0.097, 0.059, 0.013],
    [0.022, 0.097, 0.159, 0.097, 0.022],
    [0.013, 0.059, 0.097, 0.059, 0.013],
    [0.003, 0.013, 0.022, 0.013, 0.003],
])
print(f"Suma współczynników jądra Gaussa: {gaussian_kernel.sum():.4f}")

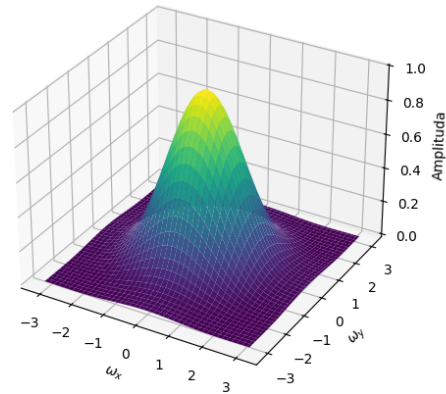
lake_gaussian = filter2d(lake, gaussian_kernel)
```

```
plot_2d_freq_response(gaussian_kernel, title="Odpowiedź częstotliwościowa ↵  
↵filtru Gaussa 5×5")  
show_row([lake, lake_gaussian], ["Oryginał LAKE", "Po filtrze Gaussa 5×5"])
```

Suma współczynników jądra Gaussa: 0.9870



Odpowiedź częstotliwościowa filtru Gaussa 5×5 — powierzchnia



Oryginał LAKE



Po filtrze Gaussa 5×5



2.3.1 Wnioski

Filtr Gaussa jest dolnoprzepustowy i symetryczny; jego odpowiedź częstotliwościowa ma kształt Gaussiany w dziedzinie częstotliwości. Współczynniki jądra sumują się do 1, więc nie zmienia się średnia jasność obrazu. Filtr skutecznie usuwa drobne szczegóły i szum, zachowując gładkie

przejścia tonalne.

2.4 Zadanie 4.

Przetwórz obraz LAKE.bmp górnoprzepustowymi filtrami wyostrającymi o jądrach 3×3 , 5×5 , 7×7 , gdzie element centralny wynosi $(1 - 1/n^2)$, a pozostałe $-1/n^2$. Wyświetl odpowiedzi częstotliwościowe i obrazy.

```
[10]: def highpass_kernel(n: int) -> np.ndarray:
        """Create n×n high-pass kernel with center (1 - 1/n²) and others -1/n²."""
        kernel = -np.ones((n, n)) / (n * n)
        center = n // 2
        kernel[center, center] = 1 - 1 / (n * n)
        return kernel

highpass_sizes = [3, 5, 7]

fig_resp, axes_resp = plt.subplots(1, 3, figsize=(15, 4))
fig_img, axes_img = plt.subplots(1, 3, figsize=(15, 4))

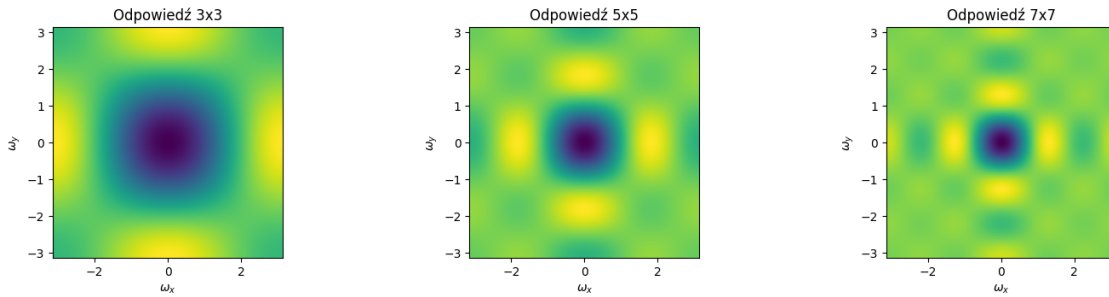
for idx, n in enumerate(highpass_sizes):
    kernel = highpass_kernel(n)
    filtered = filter2d(lake, kernel)

    axes_resp[idx].imshow(np.abs(freqz2(kernel)), cmap="viridis", extent=(-np.
    ↪pi, np.pi, -np.pi, np.pi), origin="lower")
    axes_resp[idx].set_title(f"Odpowiedź {n}x{n}")
    axes_resp[idx].set_xlabel(r"$\omega_x$")
    axes_resp[idx].set_ylabel(r"$\omega_y$")

    axes_img[idx].imshow(filtered, cmap="gray")
    axes_img[idx].set_title(f"Filtr górnoprzepustowy {n}x{n}")
    axes_img[idx].axis("off")

fig_resp.suptitle("Filtry górnoprzepustowe - odpowiedzi częstotliwościowe", y=1.
    ↪02)
fig_img.suptitle("Filtry górnoprzepustowe - obrazy po filtracji", y=1.02)
fig_resp.tight_layout()
fig_img.tight_layout()
plt.show()
```

Filtry górnoprzepustowe — odpowiedzi częstotliwościowe



Filtry górnoprzepustowe — obrazy po filtracji



2.4.1 Wnioski

Filtry górnoprzepustowe tłumią niskie częstotliwości i wzmacniają krawędzie oraz drobne szczegóły. Zwiększenie rozmiaru jądra powoduje węższą odpowiedź przestrzenną i silniejsze efekty pierścieniowe. Suma współczynników wynosi 0, dlatego obszary o stałej jasności są mapowane na wartości bliskie zero.

2.5 Zadanie 5.

Jakim filtrem jest jądro $A = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$, a jakim operator Laplasjanu $B = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$? Wyświetl odpowiedzi częstotliwościowe i skomentuj wyniki.

```
[11]: kernel_a = np.array([[0, -1, 0],
                          [-1, 5, -1],
                          [0, -1, 0]])
kernel_b = np.array([[0, -1, 0],
                     [-1, 4, -1],
                     [0, -1, 0]])

print(f"Suma jądra A: {kernel_a.sum()}")
print(f"Suma jądra B: {kernel_b.sum()}")
```

```

fig, axes = plt.subplots(1, 2, figsize=(12, 5))
axes[0].imshow(np.abs(freqz2(kernel_a)), cmap="viridis", extent=(-np.pi, np.pi,
↪ -np.pi, np.pi), origin="lower")
axes[0].set_title("Odpowiedź częstotliwościowa - jądro A")
axes[0].set_xlabel(r"$\omega_x$")
axes[0].set_ylabel(r"$\omega_y$")

axes[1].imshow(np.abs(freqz2(kernel_b)), cmap="viridis", extent=(-np.pi, np.pi,
↪ -np.pi, np.pi), origin="lower")
axes[1].set_title("Odpowiedź częstotliwościowa - Laplasjan B")
axes[1].set_xlabel(r"$\omega_x$")
axes[1].set_ylabel(r"$\omega_y$")

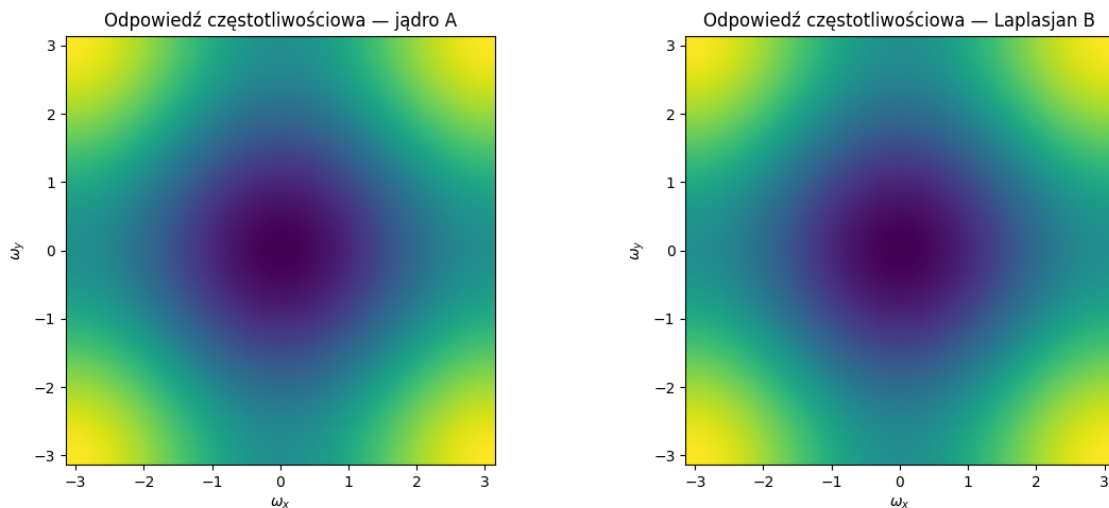
plt.tight_layout()
plt.show()

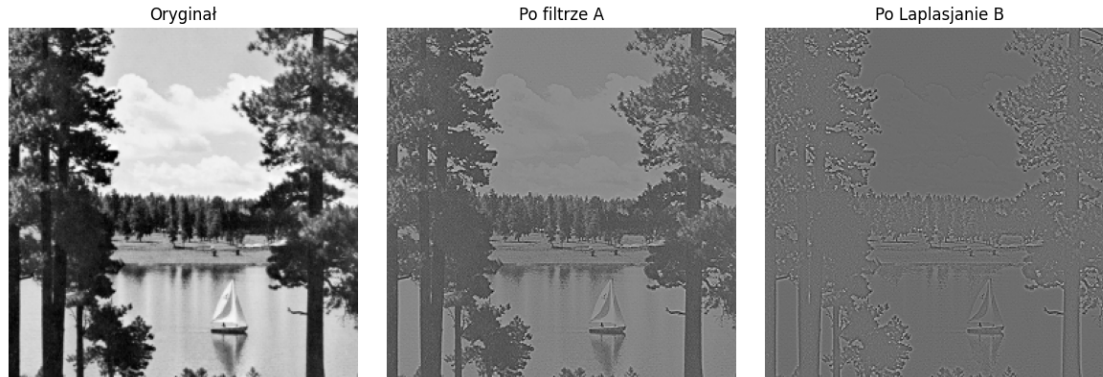
lake_a = filter2d(lake, kernel_a)
lake_b = filter2d(lake, kernel_b)
show_row([lake, lake_a, lake_b], ["Oryginał", "Po filtrze A", "Po Laplasjanie
↪ B"])

```

Suma jądra A: 1

Suma jądra B: 0





2.5.1 Wnioski

Jądro A jest filtrem wyostrzającym (unsharp mask): środkowy element o wartości 5 dodaje oryginalny obraz do wyniku Laplasjanu, co wzmacnia krawędzie bez utraty informacji o niskich częstotliwościach. Jądro B to klasyczny operator Laplasjanu (suma 0), który wykrywa drugą pochodną przestrzenną i wzmacnia szybkie zmiany jasności, tłumiąc jednocześnie obszary jednorodne.

2.6 Zadanie 6.

Obraz camels.bmp to oryginał, a camelsNOISE.bmp to wersja zaszumiona szumem pieprz-sól. Usuń szum za pomocą filtrów uśredniających 3×3 i 5×5 (c) oraz filtrów medianowych 3×3 i 5×5 (d). Wyświetl i porównaj wyniki.

```
[12]: camels = np.array(Image.open(asset_path('camels.bmp')).convert('L'))
      camels_noise = np.array(Image.open(asset_path('camelsNOISE.bmp')).convert('L'))

      avg3 = filter2d(camels_noise, averaging_kernel((3, 3)))
      avg5 = filter2d(camels_noise, averaging_kernel((5, 5)))
      med3 = median_filter(camels_noise, size=3)
      med5 = median_filter(camels_noise, size=5)

      fig, axes = plt.subplots(2, 3, figsize=(15, 10))
      axes[0, 0].imshow(camels, cmap="gray")
      axes[0, 0].set_title("Oryginał camels")
      axes[0, 0].axis("off")
      axes[0, 1].imshow(camels_noise, cmap="gray")
      axes[0, 1].set_title("camelsNOISE")
      axes[0, 1].axis("off")
      axes[0, 2].imshow(avg3, cmap="gray")
      axes[0, 2].set_title("Uśredniający 3×3")
      axes[0, 2].axis("off")
      axes[1, 0].imshow(avg5, cmap="gray")
      axes[1, 0].set_title("Uśredniający 5×5")
      axes[1, 0].axis("off")
```

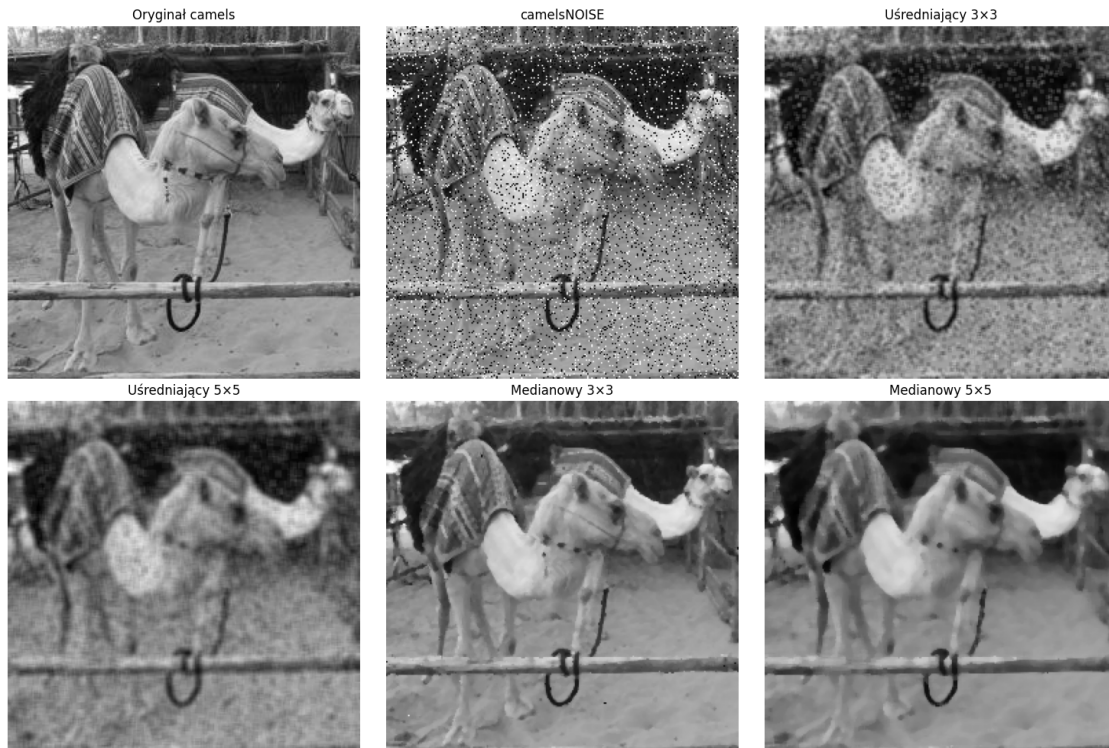


```

axes[1, 1].imshow(med3, cmap="gray")
axes[1, 1].set_title("Medianowy 3×3")
axes[1, 1].axis("off")
axes[1, 2].imshow(med5, cmap="gray")
axes[1, 2].set_title("Medianowy 5×5")
axes[1, 2].axis("off")

plt.tight_layout()
plt.show()

```



2.6.1 Wnioski

Filtry uśredniające redukują szum pieprz-sól, ale jednocześnie rozmywają krawędzie i drobne szczegóły; większe jądro daje silniejsze rozmycie. Filtry medianowe znacznie lepiej radzą sobie z szumem impulsowym, ponieważ nie uśredniają wartości skrajnych, lecz wybierają medianę. Mediana 5×5 usuwa więcej szumu niż 3×3, ale może nieco uprościć struktury obrazu.